

Assignment # 01

Title:

A* Star Algorithm

Submitted to :

- **Instructor :** Ma'am Khaula Qadeer
- **Course :** Artificial Intelligence (CSC-262)

Submitted by:

- **Name :** Sannan Ahmad
- **Reg #** (SP25-BSE-118)
- **Section** A

Date of Submission:

20 October , 2026

Question 1 (Part A): A* Search on a Grid

Formula Used

A* chooses the node with the lowest value of:

$$f(n) = g(n) + h(n)$$

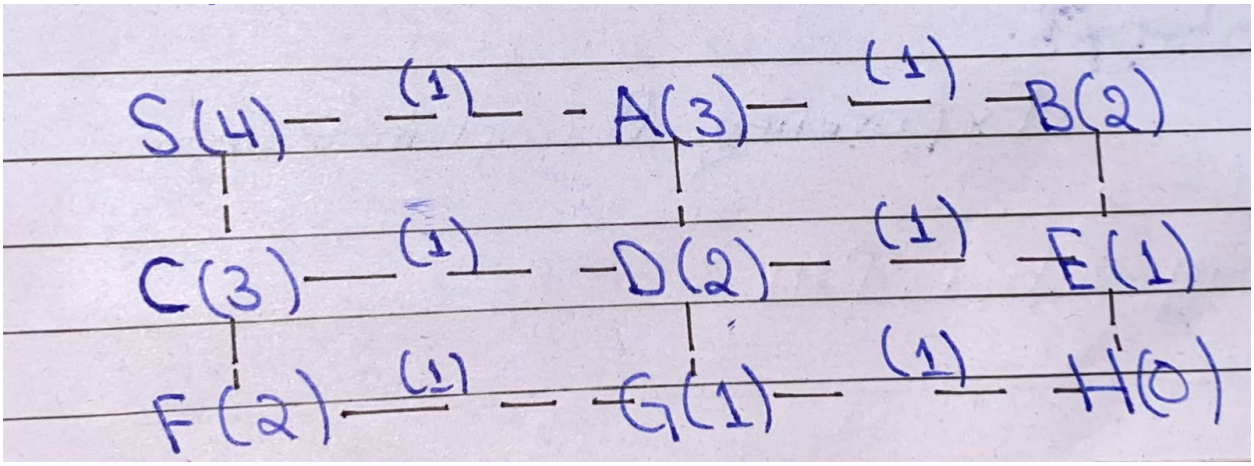
1. **f(n)** is the estimated total cost of the path through node n.
2. **g(n)** is the actual cost from the start node to node n.
3. **h(n)** is the estimated cost from node n to the goal node.

Assumptions

4. The robot moves **up, down, left and right** only (no diagonal moves).
5. Each move costs **1 unit**.
6. If two nodes tie, pick the one with the lower h. If they still tie, pick alphabetically.

Graph

The number in brackets is **h(n)**. The number on each line is the cost of that move. The **start node is S** and the **goal node is H**.



Solution

- ### Step 1: Expand S

f(S) = g(S) + h(S) = 0 + 4 = **4**. The new nodes are A and C.

A: g = 1, h = 3, f = 4

C: g = 1, h = 3, f = 4

Open list: A, C

- ### Step 2: Expand A

A and C tie, so we pick **A** alphabetically.

B: g = 2, h = 2, f = 4

D: g = 2, h = 2, f = 4

Open list: B, D, C

- ### Step 3: Expand B

B and D tie, so we pick **B** alphabetically.

E: g = 3, h = 1, f = 4

Open list: E, D, C

- ### Step 4: Expand E

We pick **E** because it has f = 4 and the lowest h (1).

H: g = 4, h = 0, f = 4

D can also be reached from E, but that gives g = 4. This is worse than D's existing g = 2, so we ignore it.

Open list: H, D, C

- ### Step 5: Reach the goal

We pick **H** because it has the lowest h (0). H is the goal, so the algorithm stops.

Result

> **Path selected by A*:** S → A → B → E → H

> **Total cost: 4**

Question 1, Part B: Consistent and Admissible Heuristics

Definitions

Admissible: h(n) is never more than the true cost from n to the goal.

Consistent: h(n) ≤ c(n, n') + h(n') for every neighbour n' of n.

Proof: Consistent ⇒ Admissible

1. Take any node n and follow the cheapest path from n to the goal: **n → n1 → n2 → ... → goal**.
2. **Apply consistency** at each step along the path:

- h(n) ≤ c(n, n1) + h(n1)
- h(n1) ≤ c(n1, n2) + h(n2)
- And so on, until the goal.

3. **Chain them together.** The step costs add up to the cost of the whole path, and the last term is h(goal) = 0.
4. **Conclude:** h(n) ≤ cost of the cheapest path = true cost.

> **Hence, every consistent heuristic is admissible.**

Example: Admissible but Not Consistent

```
A --1--> B --3--> G
```

Node	True cost to G	h
A	4	4
B	3	1
G	0	0

7. **Admissible:** 4 ≤ 4, 1 ≤ 3 and 0 ≤ 0. All are true.
8. **Not consistent:** for the edge A → B, consistency needs h(A) ≤ 1 + h(B), which means 4 ≤ 2. This is **false**.

> **Conclusion:** this h is admissible but not consistent.

Question 1, Part C: Overestimating Heuristic and the C* + c Bound

Heuristic that Overestimates

> **h(n) = 3 × (sum of Manhattan distances of all tiles)**

Plain Manhattan distance never overestimates, but multiplying it by 3 can. For example, a state that is 1 move from the goal has Manhattan distance 1, so h = 3. This is more than the true cost of **1**.

How It Gives a Sub-optimal Solution

Take a state with two routes to the goal. The h values below are assumed to show the idea.

```
Start --1--> X --1--> Goal    (total cost = 2, optimal)
Start --1--> Y --3--> Goal    (total cost = 4)
```

Node	g	h (given)	f = g + h
X	1	5 (true cost is 1)	6
Y	1	1	2

- A* expands **Y** first, because f(Y) = 2 is lower than f(X) = 6.
- The goal reached through Y has f = 4, which is still lower than f(X) = 6.
- So A* picks that goal and returns **cost 4**.

The optimal cost is 2, so the answer is sub-optimal.

Proof: If h Overestimates by at Most c, A* Returns Cost ≤ C* + c

Given: $h(n) \le h^*(n) + c$ for every node n, where $h^*(n)$ is the true cost. Let **C*** be the optimal cost.

- Say A* returns a bad goal G2 whose cost is more than $C^* + c$.
- The best path is still partly unexplored, so some node n from it is waiting in the open list.
- For that node, **$f(n) \le C^* + c$**, because h(n) is at most c above the true cost. In short, $f(n) = g(n) + h(n) \le g(n) + h^*(n) + c = C^* + c$.
- But f(G2) is more than $C^* + c$, so f(n) is smaller than f(G2).
- A* picks the smallest f, so it would choose n before G2. This is a contradiction.

Hence, A* never returns a cost above C* + c.

FIN !